

Análise Crítica de Artigo Científico sobre Testes de Software

Antonio Matheus Pereira de Lima¹, Fernanda Vitória Araújo Santos²,
João Guilherme da Silva³, Juan do Monte Pereira⁴

¹Análise em Desenvolvidores de Sistemas – Instituto Federal do Piauí (IFPI)
Caixa Postal gabinete.capic@ifpi.edu.br – 64605-5

Resumo. Este trabalho apresenta um resumo crítico do artigo científico *Automated NFR testing in continuous integration environments: a multi-case study of Nordic companies*, publicado em 2023 no periódico *Empirical Software Engineering*. São discutidos o problema abordado, os objetivos, a metodologia, a solução proposta, os principais resultados, as limitações e a aplicação prática da abordagem. Além disso, é apresentada uma demonstração prática baseada nos conceitos discutidos pelos autores.

1. Introdução

Este trabalho apresenta um resumo crítico do artigo científico *Automated NFR testing in continuous integration environments: a multi-case study of Nordic companies*, de autoria de Liang Yu, Emil Alégroth, Panagiota Chatzipetrou e Tony Gorschek, publicado no periódico *Empirical Software Engineering* (volume 28, artigo 144). O artigo foi aceito em 7 de junho de 2023 e disponibilizado on-line em 24 de outubro do mesmo ano. Os autores estão vinculados ao Blekinge Institute of Technology (Suécia), à Örebro University School of Business (Suécia) e à Fortiss GmbH (Alemanha).

O estudo se insere na área de Engenharia de Software, com foco em testes automatizados de Requisitos Não Funcionais (*Non-Functional Requirements* – NFRs) em ambientes de Integração Contínua (*Continuous Integration* – CI). Trata-se de um tema atual e relevante, uma vez que a automação de testes funcionais já é uma prática consolidada na indústria de software, enquanto a automação de testes de NFRs ainda representa um desafio significativo para a indústria de software.

2. Problema Estudado

O artigo investiga como habilitar e apoiar a realização de testes automatizados de Requisitos Não Funcionais dentro de ambientes de Integração Contínua. Requisitos como desempenho, segurança, confiabilidade, escalabilidade e manutenibilidade influenciam diretamente a qualidade e o sucesso de um produto de software, além de frequentemente precisarem atender a regulamentações específicas, como o *General Data Protection Regulation* (GDPR).

Apesar dessa importância, muitas organizações ainda dependem de práticas predominantemente manuais para validar NFRs, o que torna o processo menos eficiente, podendo criar gargalos nos ambientes de CI e aumentar a probabilidade de que problemas sejam identificados apenas nas fases finais do desenvolvimento. Segundo os autores, defeitos relacionados a requisitos não funcionais podem gerar retrabalho equivalente a 40% a 50% do esforço total de um projeto, elevando custos e comprometendo a qualidade do

software entregue. Esse cenário evidencia uma lacuna prática e teórica que justifica a realização do estudo.

3. Objetivo

O objetivo principal do artigo é examinar como o teste automatizado de Requisitos Não Funcionais pode ser habilitado e suportado por ambientes de Integração Contínua em empresas de desenvolvimento de software. Para alcançar esse objetivo, os autores formularam quatro perguntas de pesquisa (*Research Questions* – RQs):

- **RQ1:** Quais tipos de NFRs são verificados por meio de testes automatizados?
- **RQ2:** Quais métricas são utilizadas para testes automatizados de NFRs na prática industrial?
- **RQ3:** Como os ambientes de Integração Contínua contribuem para as capacidades de teste de NFRs?
- **RQ4:** Quais desafios estão associados aos testes automatizados de NFRs?

Essas perguntas delimitam progressivamente o escopo da investigação: partem de um levantamento descritivo (tipos de NFR e métricas empregadas) e avançam para uma análise mais aprofundada sobre a contribuição efetiva dos componentes de CI e os desafios práticos enfrentados pelas equipes.

4. Metodologia

A pesquisa foi conduzida por meio de um estudo de caso múltiplo, com o objetivo de investigar como empresas de desenvolvimento de software utilizam ambientes de Integração Contínua para apoiar a automação de testes de NFRs. O processo metodológico foi dividido em quatro fases:

1. Planejamento, com definição dos objetivos e das questões de pesquisa;
2. Elaboração do desenho do estudo de caso, incluindo roteiro das entrevistas, seleção dos participantes e testes-piloto;
3. Coleta de dados por meio de entrevistas semiestruturadas;
4. Análise temática e síntese dos dados coletados.

A coleta de dados foi realizada por meio de 22 entrevistas semiestruturadas com profissionais da indústria de software, provindos de quatro empresas: *Qvantel Sweden*, *Ericsson*, *Fortnox* e uma empresa anonimizada por questões de confidencialidade, identificada como *Company Alpha*. Ao todo, foram analisados cinco projetos distintos (Projetos A, B, C, D e E), abrangendo domínios como finanças, telecomunicações e bem-estar social. Após a transcrição das entrevistas, os dados foram analisados por meio da técnica de análise temática, método da Engenharia de Software para identificação de padrões qualitativos.

5. Solução Proposta

A solução proposta pelos autores consiste em um modelo teórico-empírico que demonstra como os ambientes de Integração Contínua podem habilitar e dar suporte à automação dos testes de NFRs. O modelo mostra como os dados produzidos durante o processo de desenvolvimento são transformados em métricas capazes de avaliar continuamente a qualidade desses requisitos.

Como entradas, o modelo recebe informações produzidas durante o desenvolvimento, como alterações no código-fonte, *commits* realizados no sistema de controle de versão e artefatos gerados durante o processo de *build*. Como saídas, o ambiente de CI produz métricas de qualidade, relatórios de vulnerabilidades, indicadores de desempenho, tendências de qualidade ao longo do tempo e notificações automáticas que auxiliam os desenvolvedores na identificação e correção de problemas relacionados aos requisitos não funcionais.

6. Resultados

Os resultados, conforme apresentados na Figura 1, indicam que os ambientes de CI possibilitam a automação de cinco tipos de NFRs: desempenho, segurança, escalabilidade, manutenibilidade e estabilidade. O requisito de desempenho foi observado em todos os projetos analisados, enquanto os requisitos de segurança estiveram presentes em quatro dos cinco projetos. Já escalabilidade, manutenibilidade e estabilidade apareceram com menor frequência, sugerindo que atributos internos ao sistema recebem menor prioridade do que atributos externos, mais visíveis ao cliente.

Engenharia de Software Empírica (2023) 28:144

Página 13 de 29 1444

Tabela 5 Tipos e atributos de NFR identificados (ISO/IEC-25023 2016)

NFR	Atributos	Atributo interno ou externo	Nome do projeto
Manutenibilidade	Testabilidade Capacidade de alteração Capacidade de modificação	Interno	Projeto B
Segurança	Vulnerabilidade Confidencialidade Autenticação Controle de acesso	Externo	Projetos A, B, C e D
Desempenho	Tempo de resposta Precisão Utilização de recursos	Externo	Projetos A, B, C, D e E
Estabilidade	Tolerância a falhas Recuperabilidade	Interna	Projeto B
Escalabilidade	Alta disponibilidade	Externo	Projetos A e B

Figura 1. Tipos e atributos de NFR identificados. (Tabela 5 do artigo)

Fonte: Yu et al. (2023).

Quanto às métricas, foram identificadas o Tempo Médio de Resposta (*Mean Response Time* – MRT), a Utilização do Processador (*Processor Usage* – PU) e a Precisão das Solicitações de API (*Accuracy of API Requests* – AOAR) para desempenho, além de métricas de criptografia de dados e avaliação de vulnerabilidades para segurança. Essas

métricas são obtidas a partir de dados coletados pelos próprios componentes do ambiente de CI, como sistemas de controle de versão, servidores de CI, ferramentas de automação de testes, plataformas em nuvem e sistemas de rastreamento de problemas.

→ **Tabela 6** Métricas de NFR extraídas para testes automatizados de NFR nos projetos industriais estudados

Tipo de NFR	Métricas de NFR	Dados nos projetos estudados	Componentes de CI relacionados	Nome do projeto
Desempenho	Tempo médio de resposta (MRT): $MRT = (A1 + A2 + \dots + An)/n$, onde Ai é o tempo que um servidor leva para responder a uma solicitação, e n é o número de respostas medidas.	Tempo de resposta de transações de pagamento entre contas em um serviço financeiro	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto A
		Tempo de resposta do gerenciamento de assinaturas de usuários em um serviço de rede móvel	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto B
		Tempo de resposta de solicitações para criar usuários e contas em um sistema bancário	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto C
	Utilização do processador (PU): $PU = A/B$, em que A é o tempo de processador necessário para executar um conjunto de tarefas e B é o tempo de operação necessário para realizar as tarefas. Precisão das solicitações de API (AOAR): $AOAR = A/B$, em que A é o número de solicitações de API com falha e B é o número total de solicitações de API processadas por um serviço.	Tempo de processamento para gerenciar dispositivos móveis e dados em um serviço web	Servidor de CI Automação de testes Servidor de CI Automação de testes Plataforma em nuvem	Projeto D
		Taxa de precisão das solicitações de usuários em um sistema de assistência social	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes	Projeto E
Segurança	Criptografia de dados (DEC): $DEC = A/B$, em que A é o número de itens de dados do usuário criptografados corretamente e B é o número total de itens de dados que requerem criptografia.	Dados de privacidade do usuário em um sistema financeiro	Gerenciamento de código-fonte Sistema de controle de versão Servidor de CI Automação de testes Rastreamento de problemas	Projeto A

Figura 2. Métricas de requisitos não funcionais extraídas para testes automatizados nos projetos estudados. (Tabela 6 do artigo)

Fonte: Yu et al. (2023).

Os autores também constataram que a utilização de ambientes de CI proporciona vantagens claras em relação aos processos tradicionais, predominantemente manuais, entre elas a detecção precoce de problemas, o monitoramento contínuo da qualidade dos NFRs e a geração automática de *feedback* aos desenvolvedores, tornando o processo de desenvolvimento mais eficiente.

7. Limitações

Os autores reconhecem que alguns NFRs de natureza subjetiva, como a usabilidade, ainda são difíceis de automatizar e medir de forma confiável, dependendo, em grande parte, de avaliações realizadas por especialistas ou usuários. Além disso, o estudo identificou cinco desafios práticos recorrentes:

- **Interdependência entre submódulos:** alterações para atender a um NFR em um módulo podem causar falhas inesperadas em outros módulos;
- **Dificuldade na análise de causa-raiz,** decorrente da distribuição dos NFRs entre diferentes equipes;
- **Baixa visibilidade** para acompanhar o ciclo de vida de um NFR, da implementação até a versão final do software;
- **Instabilidade dos ambientes de CI,** que pode gerar falsos positivos, especialmente em testes de desempenho;

- **Dificuldade em centralizar e analisar** os relatórios de teste, por falta de clareza sobre quais métricas monitorar.

Quanto às ameaças à validade, os autores discutem quatro dimensões:

- **Validade interna:** possibilidade de interpretação incorreta das perguntas das entrevistas, mitigada por meio de testes-piloto.
- **Validade externa:** o estudo foi realizado apenas em empresas dos países nórdicos, limitando a generalização dos resultados para outros contextos.
- **Validade de construto:** alguns participantes poderiam deixar de relatar desafios reais, risco reduzido pela garantia de anonimato.
- **Validade de conclusão:** a análise temática dos dados envolve certo grau de subjetividade na interpretação dos resultados.

8. Aplicação Prática

A abordagem apresentada no artigo pode ser aplicada por empresas de desenvolvimento de software que já utilizam ambientes de Integração Contínua em seus processos, sendo especialmente útil para organizações que precisam monitorar continuamente requisitos como desempenho, segurança, escalabilidade e manutenibilidade. Pode ser empregada em aplicações web, sistemas corporativos, serviços em nuvem e outros projetos em que a qualidade dos requisitos não funcionais seja um fator crítico.

A técnica é aplicada durante todo o processo de desenvolvimento, integrada ao ambiente de CI, de modo que, a cada nova alteração incorporada ao código, os testes automatizados possam ser executados e as métricas de NFR atualizadas continuamente. No estudo, foram utilizadas ferramentas como JMeter, JUnit, Postman, ZAP, Anchore e SonarQube para a execução dos testes, além de Jenkins, Bitbucket, VMware e JFrog Artifactory para apoiar o processo de CI e o gerenciamento das atividades.

Com base nos resultados apresentados no artigo, a solução possui potencial para melhorar a automação dos testes de requisitos não funcionais em organizações que já utilizam Integração Contínua. Uma vez que ela permite o monitoramento contínuo da qualidade do software e a detecção precoce de problemas.

9. Conclusão

O artigo é bem estruturado e apresenta uma organização clara das informações. A divisão em introdução, fundamentação teórica, metodologia, resultados, discussão e conclusão facilita a compreensão do estudo. Além disso, os autores utilizam tabelas e figuras que contribuem para a interpretação dos resultados e para a apresentação do estudo proposto.

A metodologia adotada mostrou-se adequada aos objetivos da pesquisa. O estudo de caso múltiplo permitiu analisar práticas utilizadas em diferentes empresas de desenvolvimento de software, enquanto as entrevistas possibilitaram a obtenção de informações detalhadas sobre a utilização de ambientes de Integração Contínua para testes de requisitos não funcionais.

A técnica proposta pode ser aplicada em organizações que já utilizam ambientes de Integração Contínua e desejam aprimorar o monitoramento de requisitos não funcionais. A utilização do modelo pode contribuir para a identificação precoce de problemas

e para a melhoria contínua da qualidade do software, desde que a empresa possua uma infraestrutura de CI adequada.

Como sugestão de aprimoramento, seria interessante que futuras pesquisas realizassem estudos experimentais para validar o modelo em diferentes tipos de projetos e organizações. Além disso, a inclusão de empresas de outros países e setores poderia ampliar a aplicabilidade dos resultados e fornecer evidências adicionais sobre a eficácia da abordagem proposta.

10. Demonstração Prática

Como forma de ilustrar, na prática, o conceito central discutido no artigo analisado, foi desenvolvido uma pequena demonstração que reproduz, de maneira simplificada, um dos exemplos apresentados pelos autores (*Example 1*, Seção 4.2 do artigo): a medição automatizada de uma métrica de desempenho – o Tempo Médio de Resposta (*Mean Response Time – MRT*) – integrada a um *pipeline* de Integração Contínua.

10.1. Estrutura da demonstração

A demonstração é composta por três elementos:

- uma API mínima, desenvolvida em Node.js/Express, contendo uma rota simples que simula uma consulta de dados;
- um teste automatizado de desempenho, responsável por realizar múltiplas requisições a essa rota e calcular o MRT, utilizando a mesma fórmula apresentada na Tabela 6 do artigo ($MRT = (A_1 + A_2 + \dots + A_n) / n$);
- um *workflow* do GitHub Actions, que executa esse teste automaticamente a cada envio de código (*push*) ao repositório, reproduzindo o papel do “servidor de CI” descrito no modelo teórico do artigo (Figura ??).

10.2. Cenário de sucesso

No primeiro cenário, a aplicação responde dentro do limite de tempo previamente estabelecido (100 ms). O *pipeline* de CI executa o teste automaticamente, calcula o MRT com base nas requisições coletadas e conclui a execução com sucesso, sinalizando que o requisito não funcional de desempenho foi atendido. A Figura 3 apresenta a execução do *workflow* nesse cenário.

```
✓ Executar teste automatizado de NFR (Performance)

1 ▶ Run npm run test:performance
7
8 > ci-nfr-demo@1.0.0 test:performance
9 > node test/performance-test.js
10
11 == Teste automatizado de NFR: Performance ==
12 Endpoint sob teste: http://localhost:3000/produtos
13 Número de requisições (n): 20
14 Limite aceitável de MRT: 100 ms
15
16 [CI] Subindo a aplicação (source code -> execução)...
17 Servidor rodando em http://localhost:3000
18 [CI] Servidor no ar. Iniciando coleta de dados de performance...
19
20 Requisição 1/20: 33 ms
21 Requisição 2/20: 32 ms
22 Requisição 3/20: 32 ms
23 Requisição 4/20: 32 ms
24 Requisição 5/20: 31 ms
25 Requisição 6/20: 33 ms
26 Requisição 7/20: 32 ms
27 Requisição 8/20: 32 ms
28 Requisição 9/20: 32 ms
29 Requisição 10/20: 32 ms
30 Requisição 11/20: 32 ms
31 Requisição 12/20: 32 ms
32 Requisição 13/20: 33 ms
33 Requisição 14/20: 31 ms
34 Requisição 15/20: 31 ms
35 Requisição 16/20: 32 ms
36 Requisição 17/20: 31 ms
37 Requisição 18/20: 32 ms
38 Requisição 19/20: 31 ms
39 Requisição 20/20: 31 ms
40
41 == Resultado da métrica ==
42 MRT = (33 + 32 + 32 + 32 + 31 + 33 + 32 + 32 + 32 + 32 + 32 + 32 + 33 + 31 + 31 + 32 + 31 + 32 + 31 + 31) / 20
43 MRT = 31.85 ms
44
45 [CI] ✓ OK: MRT (31.85 ms) dentro do limite de 100 ms.
```

Figura 3. Execução do pipeline de CI com o teste de performance aprovado (MRT dentro do limite estabelecido).

10.3. Cenário de falha

No segundo cenário, uma latência artificial foi introduzida propositalmente no código da API, fazendo com que o MRT calculado superasse o limite de 100 ms definido no *workflow*. Como consequência, o *pipeline* de CI falha automaticamente, sinalizando à equipe de desenvolvimento que o requisito não funcional de desempenho não foi atendido – exatamente o comportamento de notificação automática discutido pelos autores como um dos principais benefícios de se utilizar ambientes de CI para testes de NFRs. A Figura 4 apresenta a execução do *workflow* nesse cenário.

```
Executar teste automatizado de NFR (Performance)

1 ▶ Run npm run test:performance
7
8 > ci-nfr-demo@1.0.0 test:performance
9 > node test/performance-test.js
10
11 == Teste automatizado de NFR: Performance ==
12 Endpoint sob teste: http://localhost:3000/produtos
13 Número de requisições (n): 20
14 Limite aceitável de MRT: 100 ms
15
16 [CI] Subindo a aplicação (source code -> execução)...
17 Servidor rodando em http://localhost:3000
18 [CI] Servidor no ar. Iniciando coleta de dados de performance...
19
20 Requisição 1/20: 153 ms
21 Requisição 2/20: 152 ms
22 Requisição 3/20: 151 ms
23 Requisição 4/20: 153 ms
24 Requisição 5/20: 152 ms
25 Requisição 6/20: 152 ms
26 Requisição 7/20: 152 ms
27 Requisição 8/20: 151 ms
28 Requisição 9/20: 152 ms
29 Requisição 10/20: 152 ms
30 Requisição 11/20: 152 ms
31 Requisição 12/20: 151 ms
32 Requisição 13/20: 152 ms
33 Requisição 14/20: 152 ms
34 Requisição 15/20: 152 ms
35 Requisição 16/20: 152 ms
36 Requisição 17/20: 152 ms
37 Requisição 18/20: 152 ms
38 Requisição 19/20: 151 ms
39 Requisição 20/20: 151 ms
40
41 == Resultado da métrica ==
42 MRT = (153 + 152 + 151 + 153 + 152 + 152 + 152 + 151 + 152 + 152 + 152 + 152 + 152 + 152 + 152 + 152 + 151 + 151) / 20
43 MRT = 151.85 ms
44
45 [CI] ✗ FALHA: MRT (151.85 ms) excedeu o limite de 100 ms.
46 [CI] O pipeline será marcado como falho para notificar a equipe.
47 Error: Process completed with exit code 1.
```

Figura 4. Execução do pipeline de CI com o teste de performance reprovado (MRT acima do limite estabelecido).

10.4. Repositório da demonstração

O código-fonte completo da demonstração, incluindo a API, o script de teste automatizado e o *workflow* do GitHub Actions, está disponível no seguinte repositório:

<https://github.com/Fernanda135/ci-nfr-demo>

Referências

Yu, L., Alégroth, E., Chatzipetrou, P., and Gorschek, T. (2023). Automated nfr testing in continuous integration environments: a multi-case study of nordic companies. *Empirical Software Engineering*, 28(6):144.